PYTHON PROGRAMMING

Simple 4-Mark Answers • B.Tech First Year

Quick guide

• Use the short bullet points for theory answers.
• For programs, learn the code and understand what each line does.
• Sample names and numbers are only examples.

UNIT I – PYTHON FUNDAMENTALS

1. Python features and interpreter

- Python is a simple, high-level programming language.
- Its code is easy to read and write.
- It works on Windows, Linux, and macOS.
- It has many ready-made tools called libraries.
- The Python interpreter reads and runs Python code.

2. Display name, branch, and semester

- Use print() to show information on the screen.

Program:

```
print("Name: Anu")
print("Branch: CSE")
print("Semester: I")
```

Sample output:

```
Output:
Name: Anu
Branch: CSE
Semester: I
```

3. Variables, identifiers, and data types

- A variable stores a value, for example: age = 18.
- An identifier is a name given to a variable, such as age.
- int stores whole numbers: 10.
- float stores decimal numbers: 3.5.
- str stores text: 'Hello'; bool stores True or False.

4. Sum and product of two numbers

- Read two numbers, then add and multiply them.

Program:

```
a = int(input("First number: "))
b = int(input("Second number: "))
print("Sum =", a + b)
print("Product =", a * b)
```

Sample output:

```
Example output (4 and 5):
Sum = 9
Product = 20
```

5. int, float, str, and bool

- int: whole number, e.g. 8.
- float: decimal number, e.g. 2.5.
- str: text, e.g. 'Ravi'.
- bool: True or False, e.g. True.
- These are common Python data types.

6. Celsius to Fahrenheit

- Formula: Fahrenheit = (Celsius × 9/5) + 32.

Program:

```
c = float(input("Celsius: "))
f = (c * 9/5) + 32
print("Fahrenheit =", f)
```

Sample output:

```
Example output (25°C):
Fahrenheit = 77.0
```

7. Arithmetic and relational operators

- Arithmetic operators do calculations: +, -, *, /, %, //, **.
- Example: 7 + 2 = 9.
- Relational operators compare values: ==, !=, >, <, >=, <=.
- They give True or False.
- Example: 5 > 3 gives True.

8. Area and perimeter of rectangle

- Area = length × breadth.
- Perimeter = 2 × (length + breadth).

Program:

```
l = float(input("Length: "))
b = float(input("Breadth: "))
print("Area =", l * b)
print("Perimeter =", 2 * (l + b))
```

Sample output:

```
Example output (5 and 3):
Area = 15.0
Perimeter = 16.0
```

9. Comments, indentation, and program structure

- A comment starts with # and explains the code.
- Indentation means spaces at the start of a line.
- Indentation shows which statements belong together.
- A simple program can take input, process it, and show output.

Program:

```
# This is a comment
name = "Ravi"
print("Hello", name)
```

Sample output:

```
Output:
Hello Ravi
```

10. Average of three numbers

- Add the three numbers and divide by 3.

Program:

```
a = float(input("First number: "))
b = float(input("Second number: "))
c = float(input("Third number: "))
avg = (a + b + c) / 3
print("Average =", avg)
```

Sample output:

```
Example output (10, 20, 30):
Average = 20.0
```

11. Implicit and explicit type conversion

- Implicit conversion: Python changes the type automatically.
- Example: 5 + 2.0 gives 7.0.
- Explicit conversion: the programmer changes the type using int(), float(), or str().
- Example: int('12') gives 12.

12. Simple interest

- Formula: Simple Interest = (Principal × Rate × Time) / 100.

Program:

```
p = float(input("Principal: "))
r = float(input("Rate: "))
t = float(input("Time: "))
si = (p * r * t) / 100
print("Simple Interest =", si)
```

Sample output:

```
Example (1000, 5, 2):
Simple Interest = 100.0
```

13. Identifiers, keywords, and literals

- Identifier: a name, e.g. total.
- Keyword: a special Python word, e.g. if, else, for.
- Do not use keywords as variable names.
- Literal: a fixed value, e.g. 10, 2.5, 'Hi', True.

14. Join first name and surname

- Use + to join the names, with a space between them.

Program:

```
first = input("First name: ")
last = input("Surname: ")
full = first + " " + last
print("Full name:", full)
```

Sample output:

```
Example output:
Full name: Anu Rao
```

15. Arithmetic, assignment, and logical operators

- Arithmetic: do calculations, e.g. + and *.
- Assignment: store or update values, e.g. = and +=.
- Logical: combine conditions, e.g. and, or, not.
- Example: x = 5; x += 2 makes x equal to 7.
- Example: 5 > 2 and 4 > 1 gives True.

16. Total cost of three items

- Cost of one item = price × quantity.
- Add the three item costs to get the total.

Program:

```
p1 = float(input("Price 1: "))
q1 = int(input("Quantity 1: "))
p2 = float(input("Price 2: "))
q2 = int(input("Quantity 2: "))
p3 = float(input("Price 3: "))
q3 = int(input("Quantity 3: "))
total = p1*q1 + p2*q2 + p3*q3
print("Total cost =", total)
```

17. Bitwise, membership, and identity operators

- Bitwise operators work on binary numbers: &, |, ^, ~, <<, >>.
- Membership operators: in and not in check whether a value is present.
- Example: 'a' in 'cat' gives True.
- Identity operators: is and is not check whether two names refer to the same object.

18. Area of a circle to 2 decimal places

- Formula: Area = 3.14159 × radius × radius.
- round(area, 2) keeps two decimal places in the rounded number.

Program:

```
r = float(input("Radius: "))
area = 3.14159 * r * r
print("Area =", round(area, 2))
```

Sample output:

```
Example output (radius = 2):
Area = 12.57
```

19. Operator precedence and associativity

- Precedence decides which operator works first.
- Multiplication happens before addition.
- Parentheses can change the order.
- Associativity decides the order when operators have the same precedence.
- Example: 2 + 3 * 4 = 14; (2 + 3) * 4 = 20.

20. Minutes into hours and remaining minutes

- 1 hour = 60 minutes.
- // finds full hours; % finds remaining minutes.

Program:

```
minutes = int(input("Minutes: "))
hours = minutes // 60
remaining = minutes % 60
print("Hours =", hours)
print("Remaining minutes =", remaining)
```

Sample output:

```
Example output (135 minutes):
Hours = 2
Remaining minutes = 15
```

UNIT II – DECISION MAKING, LOOPS AND STRINGS

1. if, if-else, and elif

- if runs code when a condition is True.
- if-else chooses between two options.
- if-elif-else checks many conditions in order.
- Spaces at the start of lines show which code belongs to each condition.

Program:

```
if marks >= 90:
    print("A grade")
elif marks >= 75:
    print("B grade")
else:
    print("C grade")
```

2. Positive, negative, or zero

- Compare the number with 0 using if-elif-else.

Program:

```
n = float(input("Enter number: "))
if n > 0:
    print("Positive")
elif n < 0:
    print("Negative")
else:
    print("Zero")
```

Sample output:

```
Example (input = -4):
Negative
```

3. for loop and while loop

- for repeats through a sequence or range.
- while repeats as long as its condition is True.
- Use for when the number of repetitions is known.
- Use while when repetition depends on a condition.

Program:

```
for i in range(3):
    print(i)

x = 0
while x < 3:
    print(x)
    x += 1
```

Sample output:

```
Both examples print:
0
1
2
```

4. Sum of first n natural numbers

- Start total at 0.
- Add each number from 1 to n.

Program:

```
n = int(input("Enter n: "))
total = 0
for i in range(1, n + 1):
    total += i
print("Sum =", total)
```

Sample output:

```
Example (n = 5):
Sum = 15
```

5. break, continue, and pass

- break stops the loop.
- continue skips the current round and moves to the next.
- pass does nothing; it is used as a placeholder.

Program:

```
for i in range(1, 5):
    if i == 2:
        continue
    print(i)
```

Sample output:

```
Output:
1
3
4
```

6. Even numbers from 2 to 20

- range(2, 21, 2) starts at 2, stops before 21, and increases by 2.

Program:

```
for i in range(2, 21, 2):
    print(i, end=" ")
```

Sample output:

```
Output:
2 4 6 8 10 12 14 16 18 20
```

7. Nested if and if-elif-else ladder

- Nested if means an if statement inside another if statement.
- It checks a second condition inside the first condition.
- An if-elif-else ladder checks several conditions one by one.
- Example: first check age, then check whether an ID is available.

Program:

```
if age >= 18:
    if has_id:
        print("Entry allowed")
```

8. Largest of three numbers

- Compare the three numbers and print the largest.

Program:

```
a = int(input("First: "))
b = int(input("Second: "))
c = int(input("Third: "))
if a >= b and a >= c:
    largest = a
elif b >= a and b >= c:
    largest = b
else:
    largest = c
print("Largest =", largest)
```

Sample output:

```
Example (4, 9, 6):
Largest = 9
```

9. range() in a for loop

- range() gives a sequence of numbers for a loop.
- range(5) gives 0, 1, 2, 3, 4.
- range(1, 5) gives 1, 2, 3, 4.
- The stop value is not included.

10. Multiplication table from 1 to 50

- Repeat i from 1 to 50 and print number × i.

Program:

```
n = int(input("Enter number: "))
for i in range(1, 51):
    print(n, "x", i, "=", n * i)
```

11. How for and while control repetition

- Both loops repeat statements.
- for goes through items in a sequence or range.
- while repeats as long as a condition is True.
- In a while loop, update the variable so the loop can stop.

12. Factorial using a loop

- Factorial means multiplying all whole numbers from 1 to n.
- Start fact at 1, then multiply by each number up to n.

Program:

```
n = int(input("Enter a positive integer: "))
fact = 1
for i in range(1, n + 1):
    fact *= i
print("Factorial =", fact)
```

Sample output:

```
Example (n = 5):
Factorial = 120
```

13. Difference between break and continue

- break ends the whole loop.
- continue skips only the current round.

Program:

```
for i in range(1, 6):
    if i == 3:
        break
    print(i)

for i in range(1, 6):
    if i == 3:
        continue
    print(i)
```

Sample output:

```
First output: 1 2
Second output: 1 2 4 5
```

14. Count digits in a positive integer

- Divide the number by 10 using // repeatedly.
- Add 1 to count each time until the number becomes 0.

Program:

```
n = int(input("Enter number: "))
count = 0
if n == 0:
    count = 1
else:
    while n > 0:
        n //= 10
        count += 1
print("Number of digits =", count)
```

Sample output:

```
Example (12345):
Number of digits = 5
```

15. String indexing and slicing

- Indexing gets one character; counting starts at 0.
- Index -1 means the last character.
- Slicing gets part of a string using text[start:stop].
- The stop position is not included.
- Example: word = 'PYTHON'; word[0] is 'P'; word[1:4] is 'YTH'.

16. Count vowels and consonants

- Vowels are a, e, i, o, u.
- Count letters in the string; vowels go into one count and other letters into consonants.

Program:

```
s = input("Enter text: ").lower()
vowels = 0
consonants = 0
for ch in s:
    if ch.isalpha():
        if ch in "aeiou":
            vowels += 1
        else:
            consonants += 1
print("Vowels =", vowels)
print("Consonants =", consonants)
```

Sample output:

```
Example (Hello):
Vowels = 2
Consonants = 3
```

17. Four string methods

- upper(): changes letters to capitals; 'hi'.upper() → 'HI'.
- lower(): changes letters to small letters; 'HI'.lower() → 'hi'.
- strip(): removes spaces from both ends; ' hi '.strip() → 'hi'.
- replace(): changes matching text; 'cat'.replace('c', 'b') → 'bat'.

18. Count words and replace spaces with hyphens

- split() separates a sentence into words.
- len() counts the words; replace(' ', '-') changes spaces to hyphens.

Program:

```
s = input("Enter sentence: ")
print("Word count =", len(s.split()))
print("Updated string =", s.replace(" ", "-"))
```

Sample output:

```
Example (I love Python):
Word count = 3
Updated string = I-love-Python
```

UNIT III – LISTS AND TUPLES

1. List indexing, append(), insert(), and remove()

- A list stores many values inside square brackets [ ].
- Indexing starts at 0; nums[0] gets the first value.
- append(x) adds x at the end.
- insert(i, x) adds x at position i; remove(x) removes the first matching x.

Program:

```
nums = [10, 20, 30]
print(nums[0])
nums.append(40)
nums.insert(1, 15)
nums.remove(20)
print(nums)
```

Sample output:

```
Output:
10
[10, 15, 30, 40]
```

2. Largest and smallest in a list

- max(list) gives the largest value.
- min(list) gives the smallest value.

Program:

```
nums = [12, 5, 28, 3, 16]
print("Largest =", max(nums))
print("Smallest =", min(nums))
```

Sample output:

```
Output:
Largest = 28
Smallest = 3
```

3. Common list methods

- append(x): adds an item at the end.
- insert(i, x): adds an item at a given position.
- remove(x): removes the first matching item.
- pop(): removes and returns an item, usually the last one.
- sort(): puts items in ascending order; reverse(): reverses their order.

4. Count even items in a list and tuple

- An even number gives remainder 0 when divided by 2.
- Check each item and increase the count when n % 2 == 0.

Program:

```
nums_list = [1, 2, 4, 7, 8]
nums_tuple = (3, 6, 9, 10, 12)
lc = 0
tc = 0
for n in nums_list:
    if n % 2 == 0:
        lc += 1
for n in nums_tuple:
    if n % 2 == 0:
        tc += 1
print("Even in list =", lc)
print("Even in tuple =", tc)
```

Sample output:

```
Output:
Even in list = 3
Even in tuple = 3
```

5. Lists and tuples

- A list uses square brackets: [1, 2, 3].
- A tuple uses parentheses: (1, 2, 3).
- A list can be changed after creation; it is mutable.
- A tuple cannot be changed after creation; it is immutable.
- Both can store many values and use indexing.

6. First, last, and length of list and tuple

- Index 0 gives the first item; index -1 gives the last item.
- len() gives the number of items.

Program:

```
a = [10, 20, 30, 40]
b = (1, 2, 3, 4, 5)
print(a[0], a[-1], len(a))
print(b[0], b[-1], len(b))
```

Sample output:

```
Output:
10 40 4
1 5 5
```

7. Tuple packing and unpacking

- Packing puts multiple values into one tuple.
- Example: student = ('Ravi', 18, 'CSE').
- Unpacking puts tuple values into separate variables.
- Example: name, age, branch = student.

8. Join two tuples

- Use + to join two tuples.

Program:

```
t1 = (1, 2, 3)
t2 = (4, 5, 6)
print(t1 + t2)
```

Sample output:

```
Output:
(1, 2, 3, 4, 5, 6)
```